Ders 14

DAĞITIK SİSTEMLER: TEMELDEN İLERİYE

CAP Teoremi ve Tutarlılık Modelleri

Bölüm 3 · Tutarlılık ve Uzlaşı

Kurs	*Bölüm*	*Tarih*
Dağıtık Sistemler: Temelden İleriye	Bölüm 3 - Tutarlılık ve Uzlaşı	8 Kasım 2026

Ders transkriptinden derlenmiştir · Kişisel kullanım içindir

İçindekiler

Bu notta yer alan başlıklar

CAP Teoremi ve Tutarlılık Modelleri 3

- Özet 3
- Problemin Tanımı 3
- CAP Teoremi 3
- Tutarlılık Modelleri Yelpazesi 4
- Quorum ile Ayarlanabilir Tutarlılık 5
- PACELC: Bölünme Yokken Ne Olur? 6
- Önemli Kavramlar 6
- Dikkat Edilmesi Gerekenler 6
- Tekrar Soruları 6

CAP Teoremi ve Tutarlılık Modelleri

Bu derste, birden fazla makineye dağıtılmış bir veri sisteminin ağ bölünmesi (network partition) yaşadığında neden aynı anda hem tutarlı hem de erişilebilir kalamayacağı ele alınıyor. CAP teoreminin gerçekte ne söylediği, sıkça yapılan yanlış yorumlar ve "güçlü" ile "nihai" tutarlılık arasındaki tasarım yelpazesi örneklerle açıklanıyor.

1. *Problemin Tanımı*

Tek bir sunucuda çalışan bir veritabanında "son yazılan değer" sorusunun tek bir cevabı vardır. Veriyi birden fazla düğüme (node) kopyaladığımızda ise her kopyanın aynı anda aynı değeri göstermesi, düğümlerin birbiriyle konuşabilmesine bağlıdır. Ağ her zaman güvenilir değildir: kablolar kopar, anahtarlar (switch) yeniden başlar, veri merkezleri arasındaki bağlantı dakikalarca kesilebilir.

Eğitmen problemi iki şube örneğiyle somutlaştırıyor: İstanbul ve Ankara'daki iki sunucu aynı hesap bakiyesini tutuyor. Aralarındaki bağlantı koptuğunda, Ankara'ya gelen bir para çekme isteğine ne yapılmalı?

- **İsteği reddetmek:** Veri tutarlı kalır, fakat kullanıcı hizmet alamaz.
- **İsteği kabul etmek:** Kullanıcı hizmet alır, fakat İstanbul'daki kopya artık farklı bir bakiye gösterir.

Üçüncü bir seçenek yoktur. CAP teoreminin özü bu ikilemdir.

EK NOT

Ağ bölünmesi yalnızca kablo kopması değildir. Aşırı yük nedeniyle yanıt veremeyen bir düğüm de diğerleri açısından "ulaşılamaz" görünür; sistem bu iki durumu birbirinden ayırt edemez.

2. *CAP Teoremi*

Üç Özellik

Harf	Özellik	Anlamı
C	Tutarlılık (Consistency)	Her okuma, en son başarılı yazmanın sonucunu ya da bir hata döndürür.
A	Erişilebilirlik (Availability)	Çalışan her düğüme gelen her istek, hata olmayan bir yanıt alır.

Harf	Özellik	Anlamı
P	Bölünme toleransı (Partition tolerance)	Düğümler arası mesajlar kaybolsa ya da gecikse bile sistem çalışmaya devam eder.

Teorem şunu söyler: bir ağ bölünmesi sırasında sistem, C ile A arasından yalnızca birini garanti edebilir.

"Üçten İkisini Seç" Yanılgısı

Teorem sıkça "üç özellikten ikisini seçersiniz" diye özetlenir. Eğitmen bu özetin yanıltıcı olduğunu vurguluyor. Gerçek bir dağıtık sistemde ağ bölünmesi bir tercih değil, er ya da geç yaşanacak bir olaydır. Dolayısıyla P'den vazgeçmek pratikte mümkün değildir. Asıl soru şudur: **bölünme olduğunda sistem tutarlılığı mı, erişilebilirliği mi feda edecek?**

Bölünme yokken ise iki özellik birlikte sağlanabilir. Bu yüzden CAP, sistemin normal çalışma anını değil, yalnızca arıza anını tarif eder.

DİKKAT

"CA sistemi" diye bir şey ancak tek düğümlü bir sistem için anlamlıdır. Birden fazla düğüm varsa ve bölünmeyi hesaba katmıyorsanız, bölünme anındaki davranışınız tanımsızdır.

CP ve AP Sistemler

CP sistemler bölünme sırasında azınlıkta kalan taraftaki istekleri reddeder. Çoğunluğu (quorum) oluşturamayan bir düğüm yazma kabul etmez. Koordinasyon servisleri ve kilit yöneticileri bu gruba girer, çünkü iki farklı istemcinin aynı kilidi aldığını sanması kabul edilemez.

AP sistemler bölünme sırasında her iki tarafın da istek kabul etmesine izin verir ve bağlantı geri geldiğinde farklılaşan kopyaları uzlaştırır. Alışveriş sepeti, beğeni sayacı, sosyal medya akışı gibi kısa süreli tutarsızlığın tolere edilebildiği senaryolar için uygundur.

İPUCU

Aynı uygulama içinde farklı veriler için farklı tercihler yapılabilir: ödeme kayıtları CP, ürün önerileri AP bir depoda tutulabilir.

3. *Tutarlılık Modelleri Yelpazesi*

CAP'teki "C", yelpazenin yalnızca en güçlü ucunu temsil eder. Eğitmen, gerçek sistemlerin bu iki uç arasında farklı noktalarda durduğunu gösteriyor.

Doğrusallaştırılabilirlik (Linearizability)

En güçlü modeldir. Sistem, sanki verinin tek bir kopyası varmış ve her işlem tek bir anda gerçekleşmiş gibi davranır. Bir yazma tamamlandıktan sonra başlayan her okuma, o yazmayı ya da daha yenisini görmek zorundadır. Bedeli gecikmedir: her işlem çoğunluk düğümlerin onayını bekler.

Nedensel Tutarlılık (Causal Consistency)

Birbirine neden-sonuç ilişkisiyle bağlı işlemlerin sırası korunur, bağımsız işlemlerin sırası ise düğümden düğüme farklı görünebilir. Örneğin bir yoruma verilen yanıt, hiçbir kullanıcıya yorumun kendisinden önce gösterilmez.

Nihai Tutarlılık (Eventual Consistency)

Yeni yazma gelmezse, tüm kopyalar *bir süre sonra* aynı değere yakınsar. "Bir süre"nin ne kadar olduğu garanti edilmez. Okuma, eski bir değer döndürebilir.

Model	Okuma garantisi	Gecikme	Bölünmede erişilebilirlik
Doğrusallaştırılabilir	Her zaman en güncel değer	Yüksek	Yok (azınlık tarafı reddeder)
Nedensel	Nedensel sıra korunur	Orta	Kısmen
Nihai	Eski değer dönebilir	Düşük	Tam

4. *Quorum ile Ayarlanabilir Tutarlılık*

Pek çok veri deposu tutarlılık seviyesini istek bazında ayarlamaya izin verir. Eğitmen bunu çoğunluk (quorum) formülüyle açıklıyor: `N` kopya sayısı, `W` bir yazmanın başarılı sayılması için onay veren düğüm sayısı, `R` bir okumanın sorguladığı düğüm sayısıdır.

Eğer `R + W > N` ise, okunan düğüm kümesi ile yazılan düğüm kümesi en az bir düğümde kesişir ve okuma en son yazmayı görür.

```
def quorum_tutarli_mi(n: int, r: int, w: int) → bool:
    """Okuma ve yazma kümeleri en az bir düğümde kesişiyor mu?"""
    return r + w > n

# 3 kopya, çoğunluk yazma ve çoğunluk okuma
print(quorum_tutarli_mi(n=3, r=2, w=2))   # True  → güncel okuma
# Hızlı okuma için R=1: okuma eski veri görebilir
print(quorum_tutarli_mi(n=3, r=1, w=2))   # False → nihai tutarlılık
```

Bu ayar, CAP ikilemini tasarım zamanında değil, istek anında yönetmeyi sağlar: kritik okumalar `R=2` ile, gösterim amaçlı okumalar `R=1` ile yapılabilir.

5. *PACELC: Bölünme Yokken Ne Olur?*

Eğitmen dersi CAP'in eksik bıraktığı soruyla kapatıyor. PACELC genişletmesi şöyle okunur: bölünme (**P**) varsa erişilebilirlik (**A**) ile tutarlılık (**C**) arasında seçim yapılır; aksi halde (**E**lse) gecikme (**L**atency) ile tutarlılık (**C**) arasında seçim yapılır. Yani sistem normal çalışırken bile, güçlü tutarlılık her isteğe ağ turu ekleyerek gecikmeyi artırır.

Önemli Kavramlar

Kavram	Açıklama
Ağ bölünmesi (network partition)	Düğümlerin bir kısmının diğerleriyle iletişim kuramadığı durum
Quorum	Bir işlemin geçerli sayılması için gereken asgari düğüm onayı
Doğrusallaştırılabilirlik	Sistemin tek kopyalıymış gibi davrandığı en güçlü tutarlılık modeli
Nihai tutarlılık	Yazmalar durursa kopyaların zamanla aynı değere yakınsaması
PACELC	Bölünme yokken de gecikme–tutarlılık ödünleşimi olduğunu ekleyen model

Dikkat Edilmesi Gerekenler

> **DİKKAT**
> CAP'i "veritabanı seçme rehberi" gibi kullanmayın. Aynı veri deposu, yapılandırmasına ve istek seviyesine göre hem CP hem AP davranabilir.

> **DİKKAT**
> Nihai tutarlılık "hiç tutarlı olmamak" değildir; ancak uygulama kodu, eski veri okuma ihtimaline göre tasarlanmalıdır (ör. idempotent işlemler, sürüm numaraları).

> **İPUCU**
> Bir sistemi değerlendirirken "bölünme olursa azınlık tarafındaki istemci ne görür?" sorusu, pazarlama metinlerinden daha net cevap verir.

Tekrar Soruları

i. CAP teoremi neden "üç özellikten ikisini seç" şeklinde özetlenmemelidir?

ii. Bir CP sistemde, ağ bölünmesi sırasında azınlıkta kalan düğüme gelen yazma isteğine ne olur?

iii. `N=5`, `W=3` iken güncel okuma garantisi için `R` en az kaç olmalıdır?

iv. Nedensel tutarlılık, nihai tutarlılıktan hangi açıdan daha güçlüdür?

v. PACELC, CAP'in hangi eksikliğini giderir?